

"Changing scope for lambda trailing-return-type" (P2036) should not be a DR

Abstract

This paper explains why changing the lambda trailing-return-type scope and the other name lookup changes for lambdas should not be retroactively applied, i.e. why the changes should not be applied as DRs. Those changes silently change the meaning of valid code, with possibly breaking results, and it's irresponsible to make such a change even with the mitigation strategies proposed in P2579, due to the silent nature of the change. P2036 and P2579 are arguably fine as sanity improvements, but they should not be applied where nobody expects them, and making them DRs doesn't really serve anybody, because existing compilers will not magically gain those DR implementations.

The problem with the DR suggestion

In the unfortunate event that P2036+P2579 still end up breaking code (with init-captures, or by changing something that wasn't const into const), we have a many-fold problem:

- An implementation might just not implement the DR, and we can't force them to.
- An implementation might implement the DR so that you still need to opt in, via `-fnew-lambda-name-lookup` or some such.
- Perfectly valid code that has been valid since C++11 or C++14 changes meaning, in possibly surprising ways.
- Some users may be stuck with specific older compiler versions but also need to use newer ones or newer standard modes, so they would need to cope with multiple different meanings of the same code, across multiple compilers.

This isn't helping anyone. None of that is stable and portable. Those who wish to have the same improved semantics won't get them, and those who wish to avoid uninvited breakage of their code won't get that either. While we could entertain a feature-testing macro for this change, it's just more straightforward to leave the pre-C++23 semantics alone, and enable this silent breaking change in C++23 mode and onwards, but not otherwise.

"But you can write code that avoids the problems, so that the semantics are the same in every conformance mode on every implementation, DR or not"

Sure, I can. If I'm aware of having to do so when I already wrote 100% well-formed, valid, and works-as-intended code until WG21 decided to change its meaning. Vice versa, it's possible to write C++20 or C++11 code that avoids the problems depicted in P2036, and both of these are arguments towards "if you know what you're doing, all's well ends well". P2036 may make such an argument, but the existing code came first. And we don't and can't know how much of possibly problematic code exists, and where.

While it's not very nice to make such a silent change in meaning to begin with, it's far worse to maybe or maybe not apply it retroactively. That leaves unaware users without any mitigation strategy, they need to become aware and change their existing code, which they thought to be valid. By not making this change a DR, they at least have a simple solution of not bumping the standard version they use. With the DR, they have no such simple solution, all they have is chaos that requires immediate action, as opposed to being able to choose when to act.

The solution

The solution is simple, do what we always do, when considering breaking changes, especially silent ones: do not apply them retroactively, don't make these changes a DR. That gives users an escape hatch, allowing them to keep their code in a working state without changing it. And since it's a change across standard versions, it gives them the ability to decide when to deal with the possible breakage, and trust not having to do so prior to that.